

INTRODUÇÃO À INTELIGÊNCIA ARTIFICIAL 25-26

07. ALGORITMOS GENÉTICOS

Cpereira_
@ISEC.PT

Índice

2

- Índice
 - Introdução
 - Funcionamento
 - Selecção
 - Recombinação
 - Mutação

Introdução

3

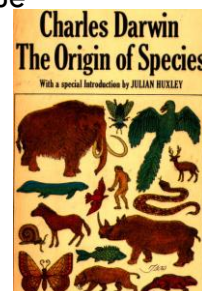
- Computação Evolucionária (CA)
 - ▣ A área de investigação designada por “Computação evolucionária (EC)” envolve:
 - Algoritmos Genéticos (GA)
 - Estratégias Evolucionárias (ES)
 - Programação Evolucionária (EP)



Introdução

4

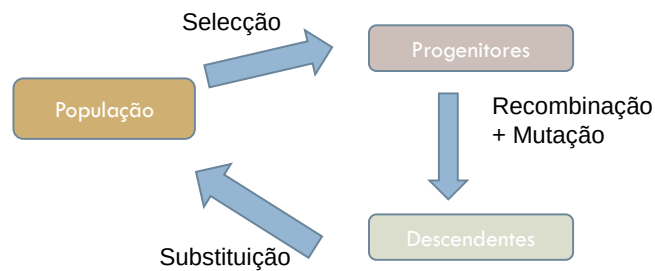
- Algoritmos Genéticos
 - ▣ Técnica para resolução de problemas que necessitem de optimização
 - ▣ Baseados na Teoria de Evolução de Darwin
 - Sub-classe da computação evolucionária.
 - A Computação evolucionária desenvolveu-se nos anos 60
 - Os GAs foram criados a meio da década de 70 por John Holland



Introdução

5

□ Princípio Básico de Funcionamento



Funcionamento

6

□ Seleção

- As “melhores hipóteses” são as de maior “aptidão”. Esta aptidão é avaliada por uma função (*fitness function*).

□ Recombinação (crossover) e Mutação:

- Em vez de procurarem sistematicamente uma solução (hipótese h), os AGs geram hipóteses sucessoras das actuais (descendência/offspring) recombinaando **propabilisticamente** as melhores hipóteses entre si, e “mutando” algumas outras.

Funcionamento

7

□ Algoritmo

- *Fitness*: Função de avaliação
- *Fitness_threshold*: Critério de Fim de Ciclo
- p = Número de hipóteses da população
- r = Fração da população a ser recombinada
- m = Mutation Rate

GA(*Fitness*, *Fitness_threshold*, p, r, m)

- *Initialize*: $P \leftarrow p$ random hypotheses
- *Evaluate*: for each h in P , compute $Fitness(h)$
- While $[\max_h Fitness(h)] < Fitness_threshold$
 1. *Select*: Probabilistically select $(1 - r)p$ members of P to add to P_s .

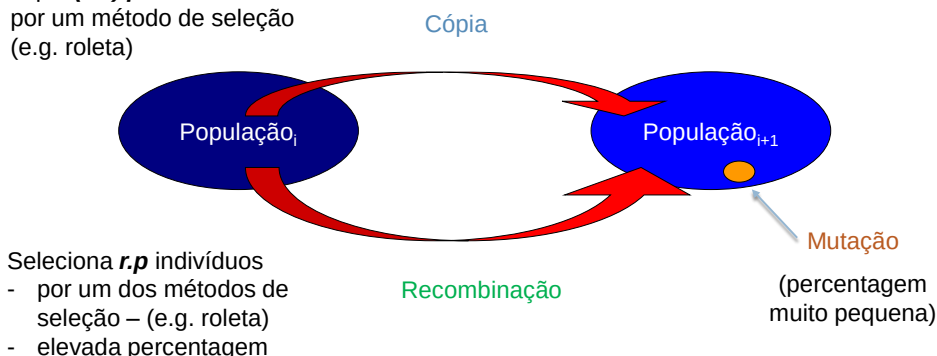
$$Pr(h_i) = \frac{Fitness(h_i)}{\sum_{j=1}^p Fitness(h_j)}$$
 2. *Crossover*: Probabilistically select $\frac{rp}{2}$ pairs of hypotheses from P . For each pair, $\langle h_1, h_2 \rangle$, produce two offspring by applying the Crossover operator. Add all offspring to P_s .
 3. *Mutate*: Invert a randomly selected bit in $m \cdot p$ random members of P_s
 4. *Update*: $P \leftarrow P_s$
 5. *Evaluate*: for each h in P , compute $Fitness(h)$
- Return the hypothesis from P that has the highest fitness.

Funcionamento

8

□ ...

copia $(1-r) \cdot p$ indivíduos
por um método de seleção
(e.g. roleta)



Funcionamento

9

□ ...

□ Função de Fitness (aptidão)

- Define o critério que avalia cada hipótese de acordo com o objectivo a atingir.
 - É a base de qualquer critério de selecção na geração seguinte.
- Exemplos:
 - Mochila: Lucro da solução
 - N-Rainhas: A função de fitness calcula a soma dos ataques que as rainhas produzem, em cada configuração.

Funcionamento

10

□ ...

□ Seleção

- Com base na função fitness, há vários métodos de seleção das hipóteses (soluções) a incluir na próxima geração:
 - $(1-r).p$ hipóteses são copiadas
 - $r.p$ hipóteses são sujeitas a recombinação produzindo outros tantos descendentes

Seleção

11

- Selecção Proporcional
 - A **probabilidade** de selecção de uma hipótese é **proporcional** ao quociente entre a sua aptidão e a soma das aptidões das restantes. (aptidão normalizada)
 - As hipóteses com maior valor de aptidão são seleccionadas mais vezes!

Seleção

12

- ...
- Exemplo:
 - a solução de maior fitness foi seleccionada duas vezes!
 - $p=4, r=0.5$

Working Sheet of a GA

The problem is to optimize $f(x)=x$

String Number	Before Crossover (generation 0)			After Fitness Prop. Selection		After Crossover (generation 1)		
	String X_i	Fitness $f(X_i)$	$\frac{f(X_i)}{\Sigma f(X_i)}$	String X_i	Fitness $f(X_i)$	Cross-Point	X_i	$f(X_i)$
1	011	3	0.25	011	3	2	111	7
2	001	1	0.08	110	6	2	010	2
3	110	6	0.50	110	6	-	110	6
4	010	2	0.17	010	2	-	010	2
Total		12			17			17
Worst		1			2			2
Average		3.00			4.25			4.25
Best		6			6			7

Creating generation 1 from generation 0 by application of selection and crossover

cópia

Seleção

13

□ Implementação da Seleção Proporcional

■ Método da Roleta

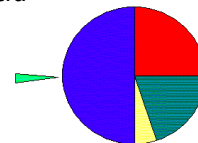
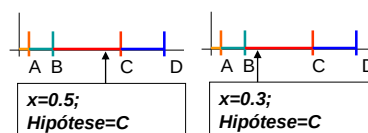
- Cada hipótese de uma dada população possui uma fitness f_i ;
 - $f_1=1/6$, $f_2=1/3$, $f_3=1$, $f_4=1/2$
- Calculam-se os valores acumulados:
 - $A=f_1=1/6$; $B=f_1 + f_2=1/2$; $C=f_1+f_2+f_3=3/2$; $D=f_1+f_2+f_3+f_4=2$
- Normalizam-se estes valores:
 - $A=0.0833$, $B=0.25$, $C=0.75$, $D=1$

Seleção

14

□ ...

- Gera-se um número aleatório x entre 0 e 1 e verifica-se sobre qual das hipóteses ele “cai”
 - Como qualquer x é igualmente provável, ele cairá mais vezes sobre a zona correspondente à hipótese que ocupa maior espaço na recta
 - Exemplos



Seleção

15

□ ...

■ Seleção por Torneio:

1. Selecionar aleatoriamente k indivíduos (k =tamanho do Torneio) de entre a população.
2. De entre elas, selecionar o de maior fitness – resultado determinístico.

■ Versão probabilística:

1. Dois indivíduos são selecionados aleatoriamente de entre a população.
2. Com uma probabilidade pré-definida (por exemplo $p=0.75$), o de maior aptidão é selecionado (o outra é selecionada com probabilidade $(1-p)$).

Seleção

16

□ ...

□ Seleção por Posicionamento (*Ranking Selection*)

- As hipóteses são ordenadas de acordo com a sua aptidão, da melhor para a pior.
- O valor do ranking (posição após a ordenação) é utilizado (em vez da aptidão) por uma função que determina a probabilidade de seleção da hipótese (o espaço que ocupará na roleta).
 - Permite evitar problemas de convergência para soluções dominantes (de aptidão muito superior às restantes).

Seleção

17

□ ...

- Linear Ranking - Atribuir uma probabilidade proporcional ao ranking, por exemplo

$$p_i = \frac{2(s-1)}{N(N-1)}(N - \text{rank}(i)) + \frac{2}{N}$$

- N - tamanho da população
- s - Pressão seletiva, valor entre 1 (probabilidade iguais) e 2 (preferência por melhores aptidões)

- Exponential Ranking - Probabilidade decresce exponencialmente com ranking

$$p_i = \frac{q(1-q)^{\text{rank}(i)-1}}{1 - (1-q)^N}$$

- q - controla a pressão seletiva, quanto maior o seu valor, maior a preferência para rankings superiores. Entre 0.2 e 0.3, considera-se uma pressão seletiva mais comum. Valores superiores a 0.7 conduz a que o melhor indivíduo seja quase sempre selecionando, eliminado a vantagem desta metodologia.

Recombinação

18

- Operadores de recombinação

- As hipóteses são, muitas vezes, representadas por strings, o que permite uma implementação simples das operações de recombinação e mutação

- Exemplos

- Problema da mochila

- “1” = objecto na mochila, “0” = objecto fora da mochila)

- 8 Rainhas

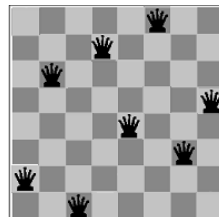
- Cada hipótese é um “estado do tabuleiro”, representado por uma string do tipo “q1q2q3q4q5q6q7q8”
- a hipótese “62714053” é uma solução “Rainha 1 = C1,L6... Rainha 8 = C8, L3”

Recombinação

19

□ ...

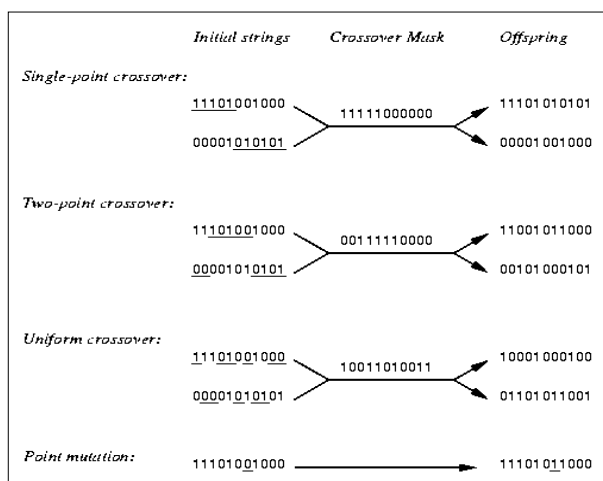
- A string “62714053” representa o genótipo (alusão ao material genético que caracteriza cada indivíduo)
- A configuração do tabuleiro representa o fenótipo (a tradução no mundo real daquilo que o genótipo determina)



Recombinação e Mutação

20

□ ...

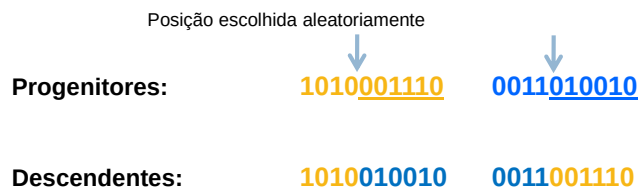


Recombinação

21

□ ...

□ Single-Point Crossover

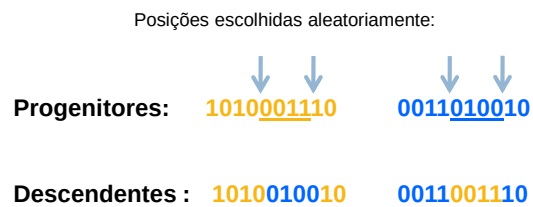


Recombinação

22

□ ...

□ Double-Point Crossover



Recombinação

23

□ ...

□ Uniform Crossover

Máscara: 0110011000 (Gerada aleatoriamente)

Progenitores: 1010001110 0011010010

Descendentes: 0011001010 1010010110

Mutação

24

□ Operadores de Mutação

Gera posição aleatória



Progenitor: 1010001110

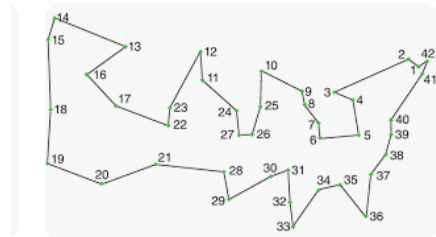
Descendente: 1010101110

Problema do Caixeiro Viajante

25

□ TSP

- ▣ Neste caso, são necessários novos operadores.
- ▣ Para a recombinação é importante não perder informação, tais como:
 - Ordem pela qual as cidades são visitadas
 - Adjacência entre cidades
 - Posição absoluta das cidades na sequência



Problema do Caixeiro Viajante

26

□ ...

▣ Recombinação por ordem

- Considerem-se os progenitores P1 e P2
- O descendente D1 é criado da seguinte forma:
 - Selecionar dois pontos de corte C1 e C2 ($C2 > C1$)
 - Copiar secção entre C1 e C2 de P1 para D1
 - Com início em C2, copiar as cidades de P2 para D1, pela ordem em que aparecem, omitindo as que já se encontram na sequência
- O Descendente D2 é criado de forma análoga

Problema do Caixeiro Viajante

27

□ ...

P1: 123**4567**89P2: 248**1395**76■ D1 **4567**...leitura de P2 "**76**248139**5**"... **281456739**

(pretende-se manter o segmento interno de P1 e preencher o restante com P2, sem duplicações e mantendo a ordem relativa).

Repetir o mesmo procedimento para obter **D2**

Nota: Existem outras variantes.

Problema do Caixeiro Viajante

28

□ ...

□ Operadores de Mutação

■ Inserção

■ l: 1 2 **3** 4 5 6 l: 1 2 4 5 **3** 6

■ Troca

■ l: 1 2 **3** 4 5 6 l: 1 2 **6** 4 5 **3**

■ Inversão

■ l: 1 2 **3** 4 5 6 l: 1 2 **4** 3 5 6